OCOM5300M Artificial Intelligence Project Performance

In [1]:
```python
# Custom Classes
from captions import Captions
from evaluation import BLEU_Evaluation
from sentence_transformers import SentenceTransformer
from rouge_score import rouge_scorer

# Deep learning Functionality
import torch  # PyTorch library

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures
import numpy as np  # numerical operations and array handling using arrays

# Natural Language Processing And Evaluation
import nltk
from nltk.translate.bleu_score import corpus_bleu, SmoothingFunction  # BLEU score evaluation
from nltk.translate.meteor_score import meteor_score
import re

# Visualization
import matplotlib.pyplot as plt  # plotting and visualizing results
import seaborn as sns  # visualisation library for statistical graphics

# Utilities
import os  # file and directory operations
import random # random number generation
```

In [2]:
```python
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```python
#detect device (CPU/GPU)
device = torch.device('cpu')
```

In [4]:
```python
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [5]:
```
# seed for reproducibility
seed = 42
```

In [6]:
```
# Python random seed
random.seed(seed)
```

In [7]:
```
# NumPy random seed
np.random.seed(seed)
```

In [8]:
```
# PyTorch random seed
torch.manual_seed(seed)
```

Out[8]:
```
<torch._C.Generator at 0x1ef98fc2d50>
```

Ground Truths

In [9]:
```
# test captions filename
test_captions_filename = "ROCOv2/test_captions.csv"
```

In [10]:
```
# instantiate test captions object
test_captions = Captions()
```

In [11]:
```
# load test captions from file
test_caption_data = test_captions.load_captions(test_captions_filename)
```

In [12]:
```
# clean captions
test_captions.clean_captions();
```

In [13]:
```
# confirm number of captions
len(test_caption_data)
```

Out[13]:
```
9927
```

In [14]:
```
test_caption_data.head()
```

Out[14]:

	ID	Caption	cleaned_caption
0	ROCOv2_2023_test_000001	CT chest axial view showing a huge ascending aortic aneurysm (*).	ct chest axial view showing a huge ascending aortic aneurysm
1	ROCOv2_2023_test_000002	Computed tomography (CT) shows floating thrombosis (white arrow)	computed tomography ct shows floating thrombosis white arrow
2	ROCOv2_2023_test_000003	Digitally subtracted angiogram demonstrates active extravasation of the superior rectal artery into the ileal-conduit (blue arrow)	digitally subtracted angiogram demonstrates active extravasation of the superior rectal artery into the ileal-conduit blue arrow
3	ROCOv2_2023_test_000004	Digitally subtracted angiogram of the IMA demonstrated cessation of flow through the proximal superior rectal artery in the region of the intersection between the artery and ureter with retained perfusion of the rectosigmoid region and resolution of active extravasation	digitally subtracted angiogram of the ima demonstrated cessation of flow through the proximal superior rectal artery in the region of the intersection between the artery and ureter with retained perfusion of the rectosigmoid region and resolution of active extravasation
4	ROCOv2_2023_test_000005	Angle measurement of a Type 1 canal.	angle measurement of a type 1 canal

Load Generated Captions

In [15]:
```
# select model architecture
architecture = "Recurrent CUI"  # Recurrent, Transformer or VLM
```

In [16]:
```
# load previously generated captions from CSV
if architecture == "Recurrent":
    generated_captions = pd.read_csv("LSTM_generated_captions.csv")

elif architecture == "Recurrent CUI":
    generated_captions = pd.read_csv("LSTM_pipeline_generated_captions.csv")

elif architecture == "VLM":
    generated_captions = pd.read_csv("VLM_generated_captions.csv")

elif architecture == "VLM CUI":
    generated_captions = pd.read_csv("VLM_pipeline_generated_captions.csv")
```

In [17]:
```
# display subset of loaded captions
generated_captions.head()
```

Out[17]:

	ID	generated_caption
0	ROCOv2_2023_test_000001	computed tomography scan of the chest showing a mass in the right lung
1	ROCOv2_2023_test_000002	ct scan of the neck showing a soft tissue mass in the right neck
2	ROCOv2_2023_test_000003	selective angiogram of the left renal artery showing a large pseudoaneurysm with a filling defect in the right renal vein
3	ROCOv2_2023_test_000004	selective angiogram of the left renal artery showing a large pseudoaneurysm with a filling defect in the right renal vein white arrow
4	ROCOv2_2023_test_000005	ultrasound image of the left hip

Diagnostics

In [18]:
```
# sample, i.e., for final run all images in the test set
sample_image_ids = (test_captions.captions["ID"].drop_duplicates().tolist())
```

In [19]:
```
# filter captions
sample = test_captions.captions.loc[test_captions.captions["ID"].isin(sample_image_ids), ["ID", "cleaned_caption"]]
```

In [20]:
```
def caption_statistics(df, column_name):

    """
    Calculate caption length statistics

    Args:
       df (pandas.DataFrame): dataframe containing caption data
       column_name (str): name of the column in the dataframe that contains captions

    Returns:
       tuple: A tuple containing two elements:
           - statistics (pandas.DataFrame): dataframe with statistical measures
             (min, median, mean, max, standard deviation) realting to caption lengths
           - caption_lengths (pandas.Series): series containing the word count for each caption
    """
```

```
    try:

        # summary statistics
        statistics = pd.DataFrame(columns=['min', 'median', 'mean', 'max', 'std'])

        # return the number of words in the caption and statistical analysis
        caption_lengths = df[column_name].apply(lambda text: len(str(text).split()))

        # add caption lengths statistics, i.e., minimum, median, mean, maximum and standard deviation
        statistics.loc[len(statistics)] = [round(caption_lengths.min(), 2),
                                           round(caption_lengths.median(), 2),
                                           round(caption_lengths.mean(), 2),
                                           round(caption_lengths.max(), 2),
                                           round(caption_lengths.std(), 2)]

        # return caption length statistics and individual caption lengths
        return statistics, caption_lengths

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [21]:

```
def format_statistics_table(df_generated_statistics, df_ground_truth_statistics):

    """
    Create HTML table to display and compare caption statistics from different caption generation methods,
    i.e., generated captions and ground truth.

    Args:
        df_generated_statistics (pandas.DataFrame): dataFrame containing statistical measures
                                                    for generated captions.
        df_ground_truth_statistics (pandas.DataFrame): dataFrame containing statistical measures
                                                       for ground truth captions.

    Returns:
        str: HTML-formatted string representing a table with caption statistics.
             The table includes rows for generated captions and ground truth,
             with columns for statistical measures (Min, Median, Mean, Max, Std).
    """

    try:
```

```
        # table header
        html_table = """
        <table border="1" class="dataframe">
          <thead>
            <tr style="text-align: center;">
              <th>Caption Type</th>
              <th>Min</th>
              <th>Median</th>
              <th>Mean</th>
              <th>Max</th>
              <th>Std</th>
            </tr>
          </thead>
          <tbody>
        """

        # format and add generated captions row
        min_val = f"{df_generated_statistics['min'].iloc[0]:.2f}" if 'min' in df_generated_statistics else ""
        median_val = f"{df_generated_statistics['median'].iloc[0]:.2f}" if 'median' in df_generated_statistics else
""
        mean_val = f"{df_generated_statistics['mean'].iloc[0]:.2f}" if 'mean' in df_generated_statistics else ""
        max_val = f"{df_generated_statistics['max'].iloc[0]:.2f}" if 'max' in df_generated_statistics else ""
        std_val = f"{df_generated_statistics['std'].iloc[0]:.2f}" if 'std' in df_generated_statistics else ""

        html_table += f"""
        <tr>
          <td>Generated Captions</td>
          <td>{min_val}</td>
          <td>{median_val}</td>
          <td>{mean_val}</td>
          <td>{max_val}</td>
          <td>{std_val}</td>
        </tr>
        """

        # format and add ground truth row

        min_val = f"{df_ground_truth_statistics['min'].iloc[0]:.2f}" if 'min' in df_ground_truth_statistics else ""
        median_val = f"{df_ground_truth_statistics['median'].iloc[0]:.2f}" if 'median' in
df_ground_truth_statistics else ""
        mean_val = f"{df_ground_truth_statistics['mean'].iloc[0]:.2f}" if 'mean' in df_ground_truth_statistics else
```

```
""
        max_val = f"{df_ground_truth_statistics['max'].iloc[0]:.2f}" if 'max' in df_ground_truth_statistics else ""
        std_val = f"{df_ground_truth_statistics['std'].iloc[0]:.2f}" if 'std' in df_ground_truth_statistics else ""

        html_table += f"""
        <tr>
          <td>Ground Truth</td>
          <td>{min_val}</td>
          <td>{median_val}</td>
          <td>{mean_val}</td>
          <td>{max_val}</td>
          <td>{std_val}</td>
        </tr>
        """

        # Close the table
        html_table += """
          </tbody>
        </table>
        """

        return html_table

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [22]:

```
def density_plots(generated_lengths, groundtruth_lengths):

    """
    Creates and displays density plots comparing the distribution of caption lengths from different caption
generation methods,
    i.e., generated captions and ground truth.

    Args:
        generated_lengths (pandas.Series or array-like): caption lengths from generated captions
        groundtruth_lengths (pandas.Series or array-like): caption lengths from ground truth data

    """

    try:
```

```
        plt.figure(figsize=(12, 6))
        sns.set_style("whitegrid")

        # plot KDE for each distribution
        sns.kdeplot(generated_lengths, label='Generated Captions', fill=True, alpha=0.3)
        sns.kdeplot(groundtruth_lengths, label='Ground Truth', fill=True, alpha=0.3)

        # add vertical lines for averages (means)
        plt.axvline(generated_lengths.mean(), color='orange', linestyle='--',
                    label=f'Generated Mean: {generated_lengths.mean():.2f}')
        plt.axvline(groundtruth_lengths.mean(), color='green', linestyle='--',
                    label=f'GT Mean: {groundtruth_lengths.mean():.2f}')

        # configure title, axis labels, legend and layout
        plt.title('Distribution of Caption Lengths')
        plt.xlabel('Caption Length (Words)')
        plt.ylabel('Density')
        plt.legend()
        plt.tight_layout()
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [23]:
```
# calculate generated caption statistics
generated_statistics, generated_lengths = caption_statistics(generated_captions, "generated_caption")
```

In [24]:
```
# calculate ground truth statistics
groundtruth_statistics, groundtruth_lengths = caption_statistics(sample, "cleaned_caption")
```

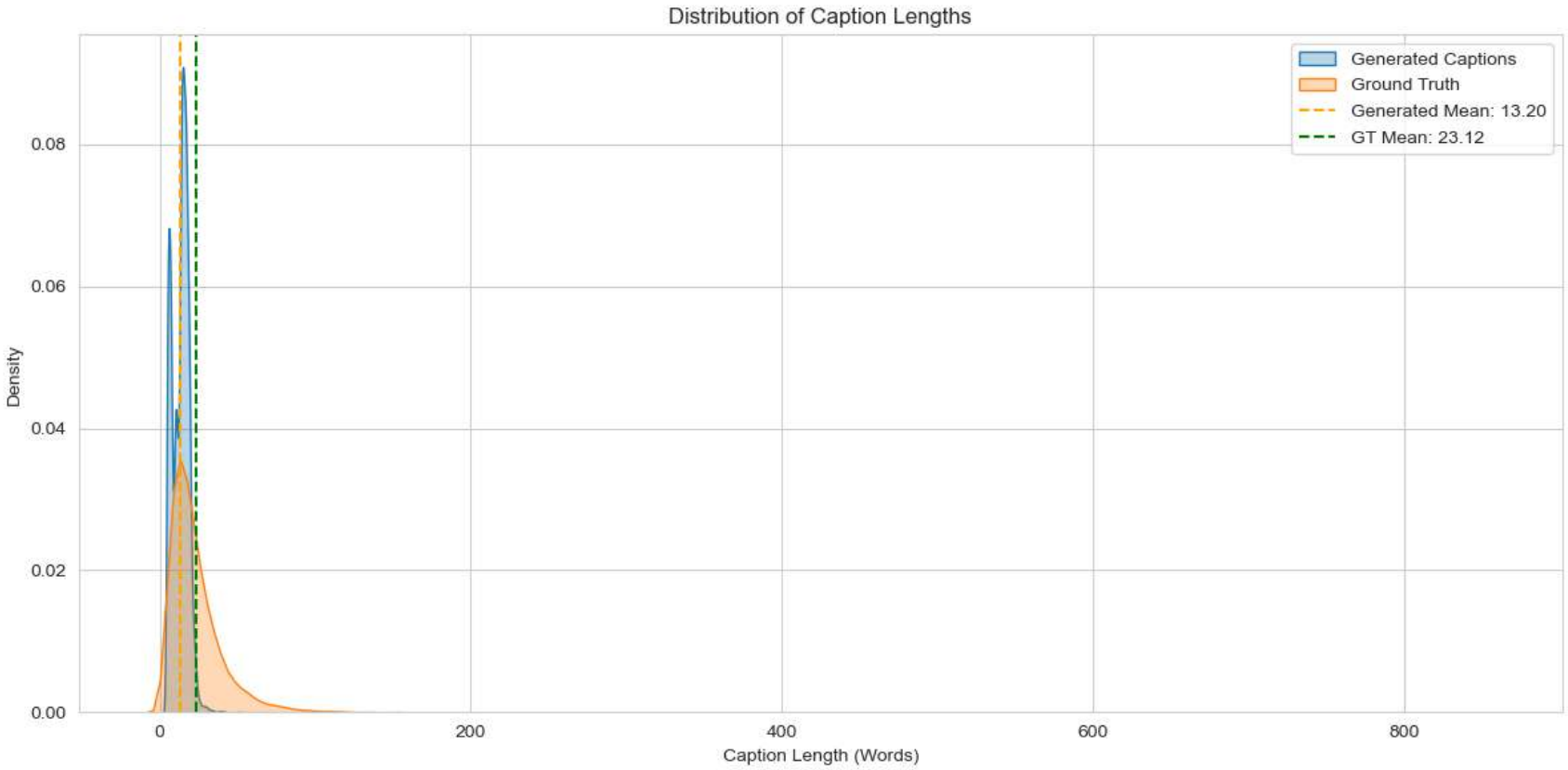
In [25]:
```
# create output
from IPython.display import HTML, display
html_output = format_statistics_table(generated_statistics, groundtruth_statistics)
```

In [26]:
```
# display statistics
HTML(html_output)
```

Out[26]:

Caption Type	Min	Median	Mean	Max	Std
Generated Captions	5.00	14.00	13.20	64.00	5.16
Ground Truth	1.00	18.00	23.12	849.00	20.19

In [27]:
```
# create density plots for caption lenghts
density_plots(generated_lengths, groundtruth_lengths)
```



Caption Evaluation Using BLEU Score

In [28]:
```
# extract captions
test_set_captions = test_captions.captions.loc[test_captions.captions["ID"].isin(sample_image_ids), ["ID",
"cleaned_caption"]]
```

In [29]:
```
# instantiate evaluator
BLEU_evaluator = BLEU_Evaluation()
```

In [30]:
```
BLEU_generated_captions = generated_captions
```

In [31]:
```
# merge test set and generated captions
BLEU_reference_generated_captions = pd.merge(test_set_captions, BLEU_generated_captions, on='ID', how='inner')
```

In [32]:
```
# calculate BLEU scores
BLEU_results = BLEU_evaluator.calculate_weighted_bleu_scores(BLEU_reference_generated_captions)
```

In [33]:
```
# display BLEU scores related to sample set
BLEU_results[BLEU_results['ID'].isin(sample_image_ids)]
```

Out[33]:

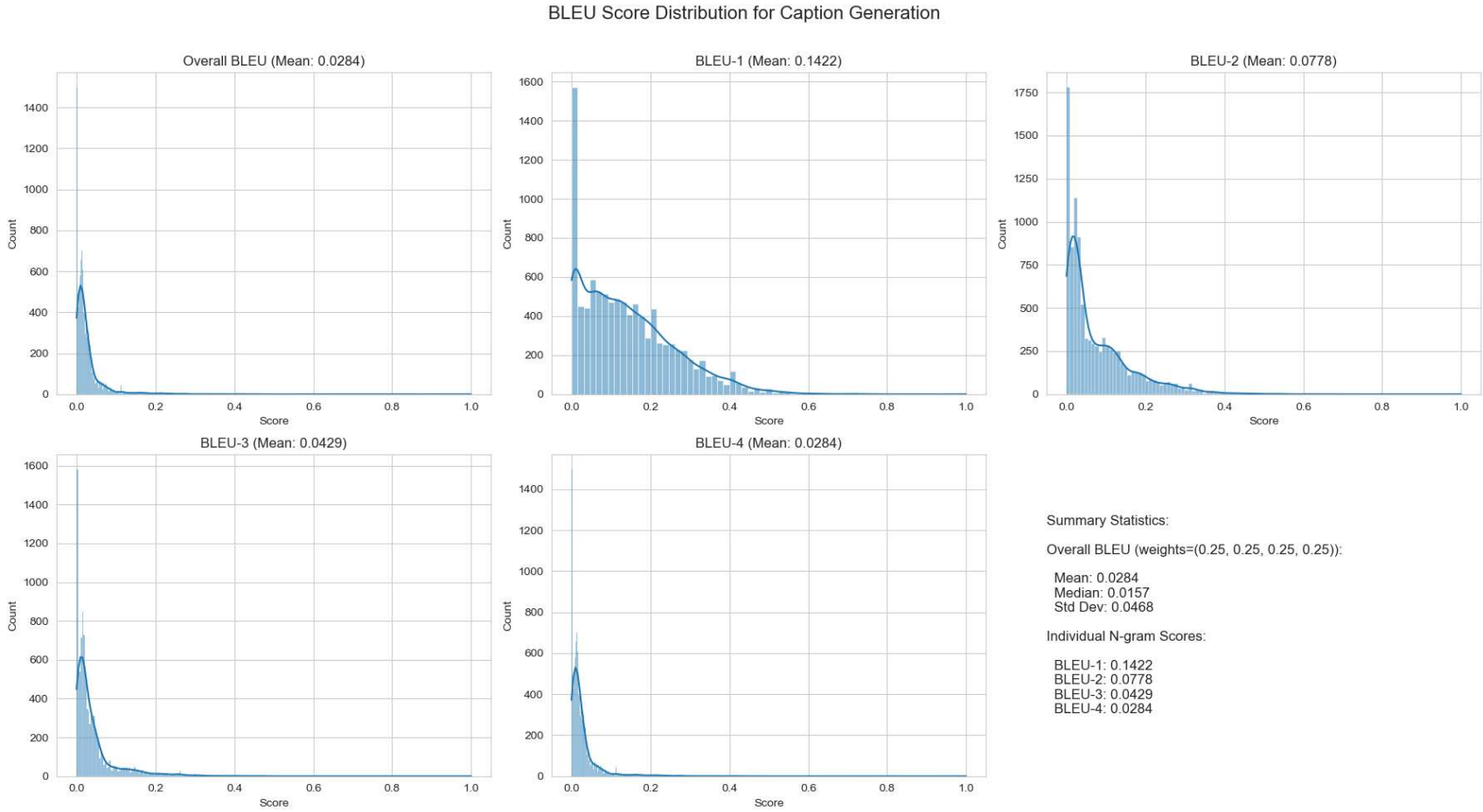
	ID	Reference Caption	Generated Caption	BLEU	BLEU-1	BLEU-2	BLEU-3	BLEU-4
0	ROCOv2_2023_test_000001	[ct chest axial view showing a huge ascending aortic aneurysm]	computed tomography scan of the chest showing a mass in the right lung	0.036362	0.230769	0.138675	0.055916	0.036362
1	ROCOv2_2023_test_000002	[computed tomography ct shows floating thrombosis white arrow]	ct scan of the neck showing a soft tissue mass in the right neck	0.014284	0.071429	0.023440	0.016605	0.014284
2	ROCOv2_2023_test_000003	[digitally subtracted angiogram demonstrates active extravasation of the superior rectal artery into the ileal-conduit blue arrow]	selective angiogram of the left renal artery showing a large pseudoaneurysm with a filling defect in the right renal vein	0.025607	0.250000	0.114708	0.041812	0.025607
3	ROCOv2_2023_test_000004	[digitally subtracted angiogram of the ima demonstrated cessation of flow through the proximal superior rectal artery in the region of the intersection between the artery and ureter with retained perfusion of the rectosigmoid region and resolution of active extravasation]	selective angiogram of the left renal artery showing a large pseudoaneurysm with a filling defect in the right renal vein white arrow	0.027155	0.146921	0.098446	0.060710	0.027155
4	ROCOv2_2023_test_000005	[angle measurement of a type 1 canal]	ultrasound image of the left hip	0.034557	0.141080	0.048872	0.036973	0.034557
...	...	...	...	...	...	...	...	...
9922	ROCOv2_2023_test_009968	[the intra-atrial septum is absent near the svc	computed tomography scan	0.004372	0.036771	0.008533	0.005385	0.004372

	ID	Reference Caption	Generated Caption	BLEU	BLEU-1	BLEU-2	BLEU-3	BLEU-4
		compatible with an svasd the blue arrow is pointing to the svasd svc : superior vena cava , svasd : sinus venosus atrial septal defect]	of the chest showing a large mass in the right atrium					
9923	ROCOv2_2023_test_009969	[papvr of a right upper lobe pulmonary vein blue arrow is seen to drain into the right svc instead of the left atrium the main and right pulmonary arteries are dilated with respect to the ascending aorta papvr : partial anomalous pulmonary venous return , svc : superior vena cava]	computed tomography scan of the chest showing a mass in the right lung	0.002853	0.022334	0.014702	0.004850	0.002853
9924	ROCOv2_2023_test_009970	[echocardiography of third week after hospitalization]	ultrasound image of the left ventricle	0.040825	0.166667	0.057735	0.043679	0.040825
9925	ROCOv2_2023_test_009971	[chest x-ray from the ed upon presentation large right sided pneumothorax with complete collapse of the right lung shift of mediastinum and heart to left consistent with tension pneumothorax proximal density left lung which may be related to tension pneumothorax or residual consolidation which was noted on patient's prior chest x-ray	chest x-ray showing bilateral pleural effusion	0.000025	0.000095	0.000073	0.000034	0.000025

	ID	Reference Caption	Generated Caption	BLEU	BLEU-1	BLEU-2	BLEU-3	BLEU-4
		ed , emergency department]						
9926	ROCOv2_2023_test_009972	[chest radiograph status post pigtail removal image taken one day before discharge previously placed pigtail catheter in the right hemithorax has been removed trachea is midline cardiac silhouette is grossly unchanged patchy opacities in the left mid and lower lung field are unchanged from prior no gross pneumothorax alveolar opacities in the right mid and lower lung fields appear stable to minimally improved suspect small right pleural effusion visualized osseous structures are grossly unremarkable]	chest x-ray showing bilateral pleural effusion	0.000001	0.000006	0.000004	0.000002	0.000001

9927 rows × 8 columns

In [34]:
```
# display BLEU score histogram
BLEU_evaluator.bleu_scores_histogram()
```



In [35]:
```
def calculate_corpus_bleu(captions_dataframe):
    """
    Calculate and display corpus BLEU-1 to BLEU-4 scores using reference and generated captions, i.e. calculate statistics across all test captions

    Args:
        captions_dataframe (pd.DataFrame): DataFrame containing image IDs, reference captions and generated captions

    Returns:
        list: corpus BLEU-1 to BLEU-4 scores
    """
```

```
    try:

        # create empty lists to store reference and generated captions for corpus-level evaluation
        corpus_references = []
        corpus_hypotheses = []

        # group captions by image ID
        for image_id, group in captions_dataframe.groupby('ID'):

            # convert reference captions to lowercase word tokens
            references = [reference.lower().split() for reference in group['cleaned_caption']]

            # convert generated caption to lowercase word tokens
            hypothesis = group['generated_caption'].iloc[0].lower().split()

            # store reference and generated captions
            corpus_references.append(references)
            corpus_hypotheses.append(hypothesis)

        # calculate corpus BLEU-1 to BLEU-4 scores
        corpus_bleu_scores = corpus_bleu(corpus_references, corpus_hypotheses, weights=[(1, 0, 0, 0),
                                                                                        (0.5, 0.5, 0, 0),
                                                                                        (1/3, 1/3, 1/3, 0), #
prevent approximation
                                                                                        (0.25, 0.25, 0.25, 0.25)],

smoothing_function=SmoothingFunction().method1)

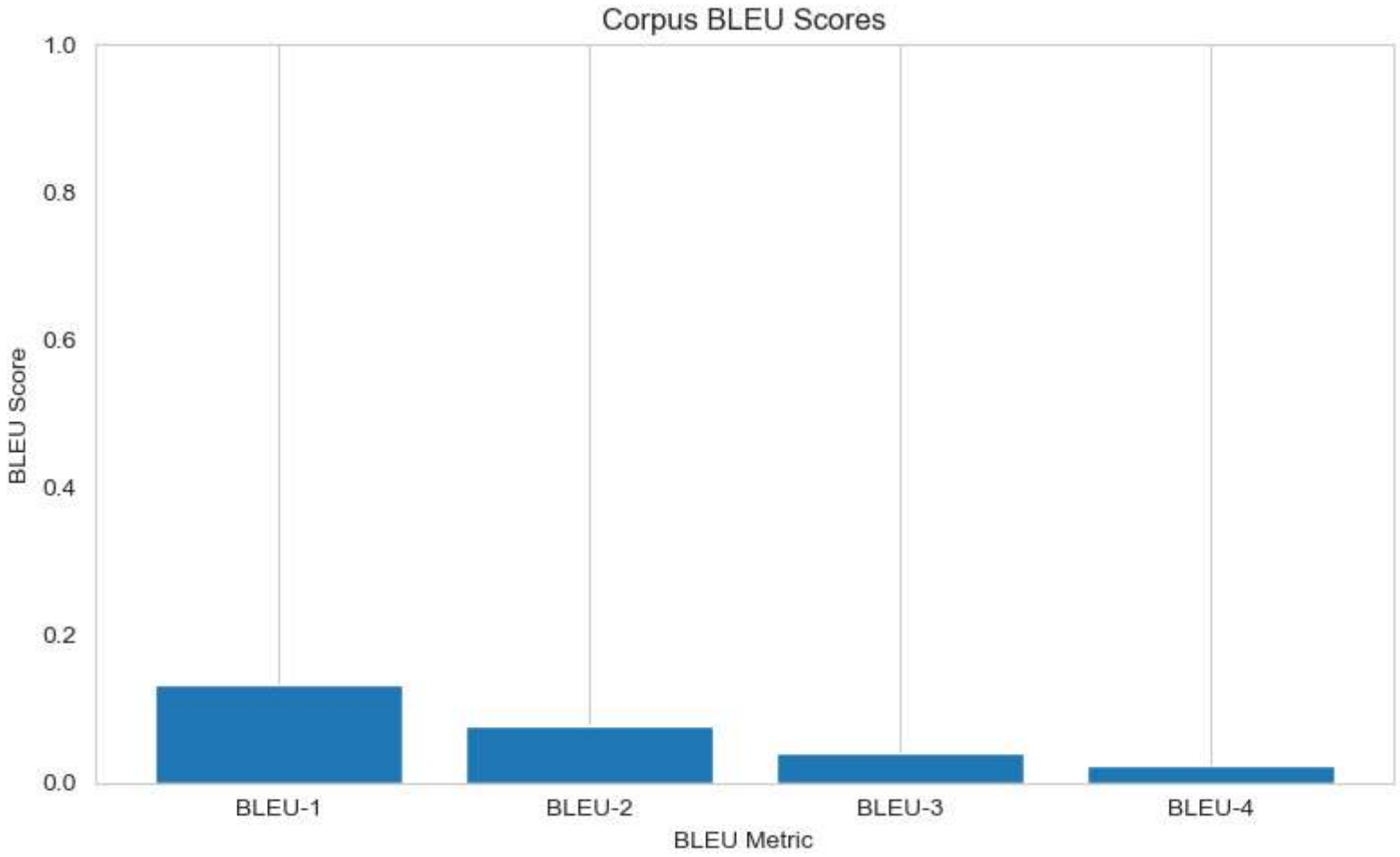

        # create BLEU score labels
        bleu_labels = ['BLEU-1', 'BLEU-2', 'BLEU-3', 'BLEU-4']

        # plot corpus BLEU scores
        plt.figure(figsize=(8, 5))
        plt.bar(bleu_labels, corpus_bleu_scores)
        plt.title('Corpus BLEU Scores')
        plt.xlabel('BLEU Metric')
        plt.ylabel('BLEU Score')
        plt.ylim(0, 1)
        plt.grid(axis='y')
        plt.tight_layout()
        plt.show()
```

```
        return corpus_bleu_scores

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [36]:
```
# calculate corpus BLEU-1 to BLEU-4 scores for the generated captions
corpus_bleu_scores = calculate_corpus_bleu(BLEU_reference_generated_captions)
```



Rouge Score

In [37]:
```
# merge ground-truth and generated captions by image ID
ROUGE_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
    how="inner",
    validate="many_to_one"
)
```

In [38]:
```
# initialise ROUGE-L evaluator
rouge_evaluator = rouge_scorer.RougeScorer(
    ["rougeL"],
    use_stemmer=False
)
```

In [39]:
```
# calculate ROUGE-L for each reference/generated caption pair
rouge_scores = [
    rouge_evaluator.score(reference, generated)["rougeL"]
    for reference, generated in zip(
        ROUGE_reference_generated_captions["cleaned_caption"],
        ROUGE_reference_generated_captions["generated_caption"]
    )
]
```

In [40]:
```
# store ROUGE-L precision, recall and F1
ROUGE_reference_generated_captions["ROUGE-L Precision"] = [
    score.precision for score in rouge_scores
]
```

In [41]:
```
ROUGE_reference_generated_captions["ROUGE-L Recall"] = [
    score.recall for score in rouge_scores
]
```

In [42]:
```
ROUGE_reference_generated_captions["ROUGE-L F1"] = [
    score.fmeasure for score in rouge_scores
]
```

In [43]:
```
# select reference with highest ROUGE-L F1 per image
best_reference_indices = (
```

```
    ROUGE_reference_generated_captions
    .groupby("ID")["ROUGE-L F1"]
    .idxmax()
)
```

In [44]:
```
# retain precision, recall and F1 from that same reference
ROUGE_results = (
    ROUGE_reference_generated_captions
    .loc[
        best_reference_indices,
        [
            "ID",
            "ROUGE-L Precision",
            "ROUGE-L Recall",
            "ROUGE-L F1"
        ]
    ]
    .reset_index(drop=True)
)
```

In [45]:
```
# display ROUGE-L summary statistics
print(
    ROUGE_results[
        ["ROUGE-L Precision", "ROUGE-L Recall", "ROUGE-L F1"]
    ].describe()
)
```

```
       ROUGE-L Precision  ROUGE-L Recall   ROUGE-L F1
count        9927.000000     9927.000000  9927.000000
mean            0.261000        0.177990     0.190257
std             0.167888        0.133437     0.118881
min             0.000000        0.000000     0.000000
25%             0.142857        0.090909     0.109589
50%             0.235294        0.150000     0.173913
75%             0.352941        0.235294     0.250000
max             1.000000        1.000000     1.000000
```

In [46]:
```
# display mean evaluation scores
print("\nMean ROUGE-L Scores")

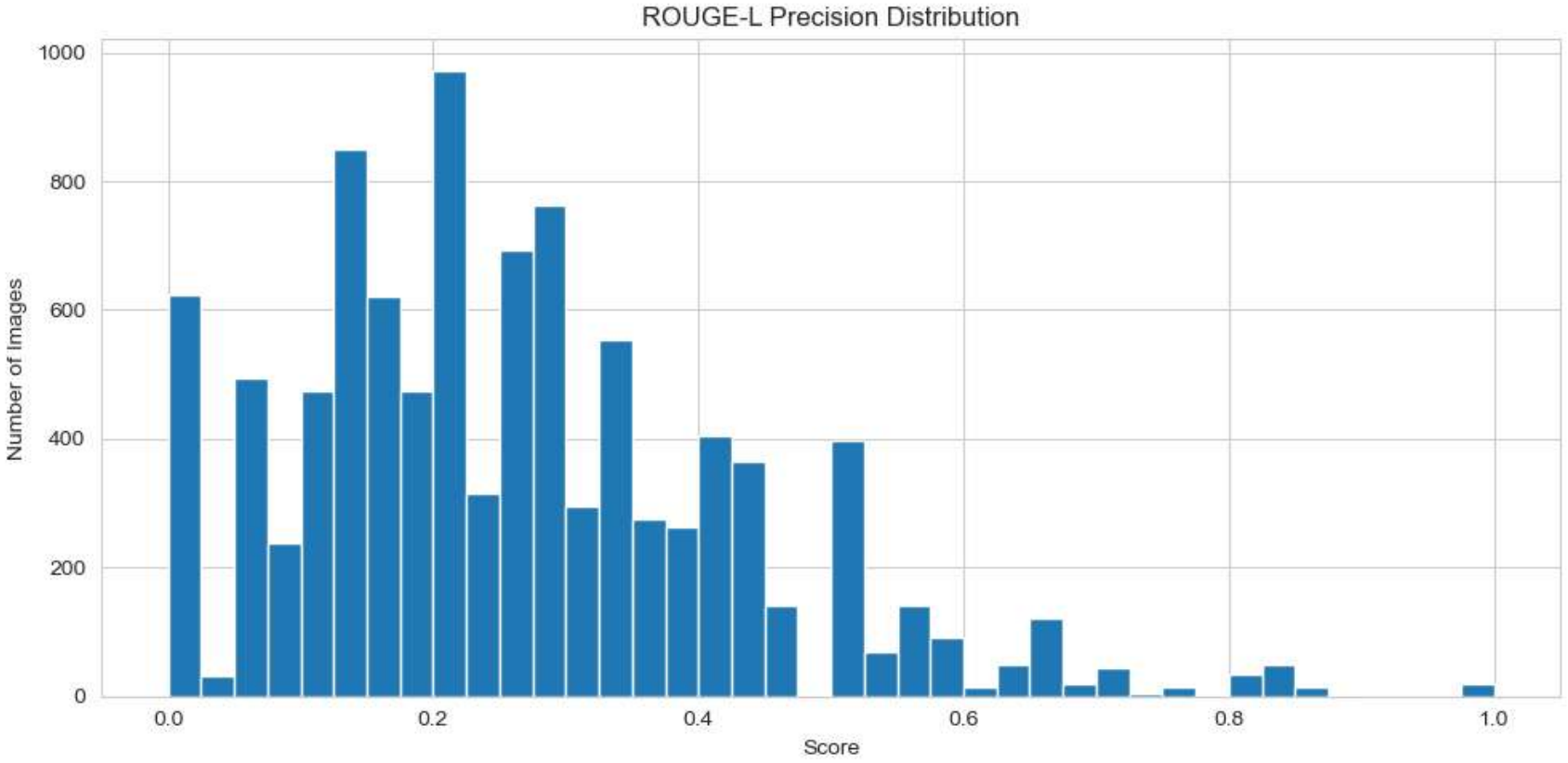
for metric in [
```

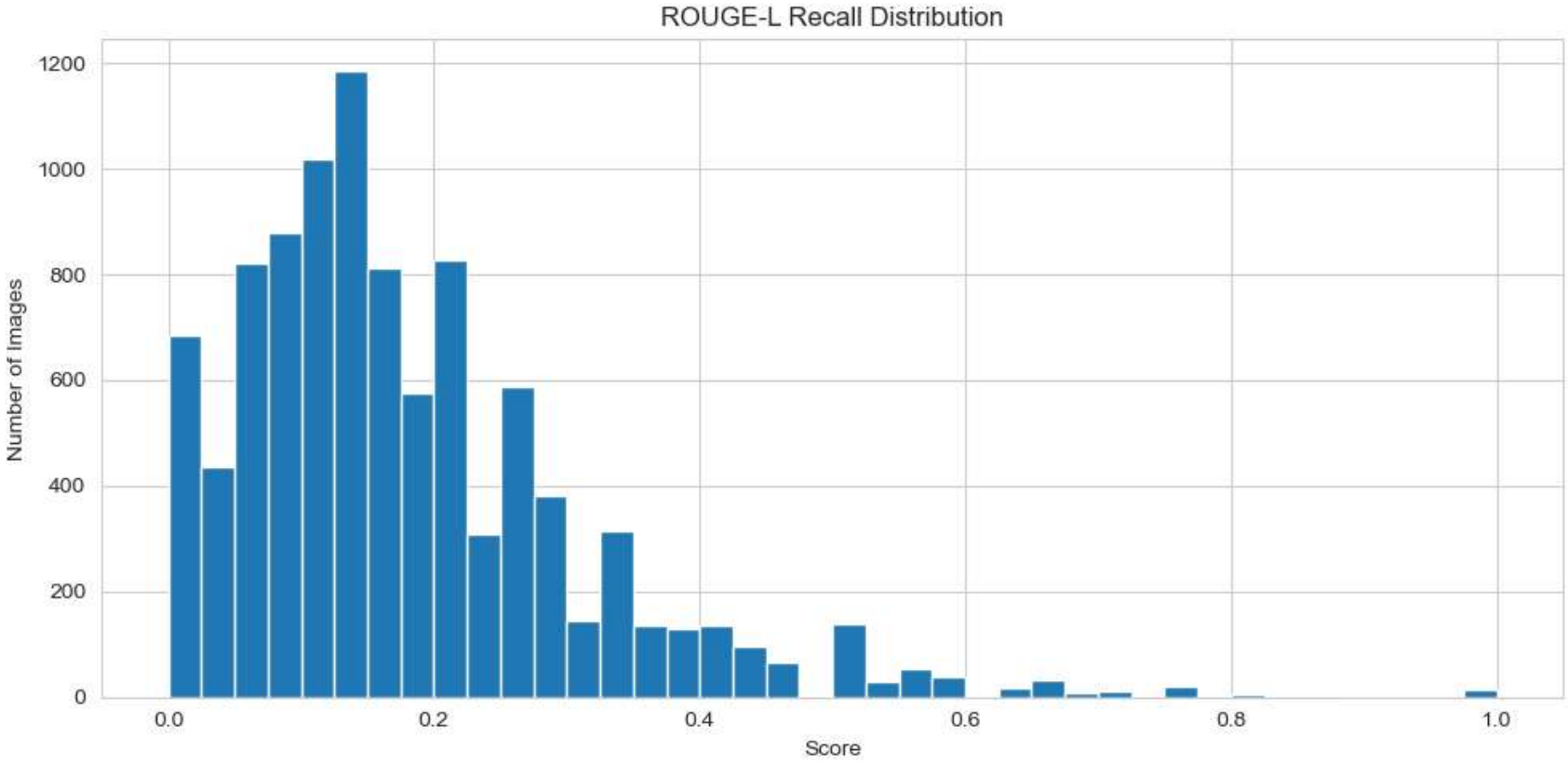
```
    "ROUGE-L Precision",
    "ROUGE-L Recall",
    "ROUGE-L F1"
]:
    print(f"{metric}: {ROUGE_results[metric].mean():.4f}")
```

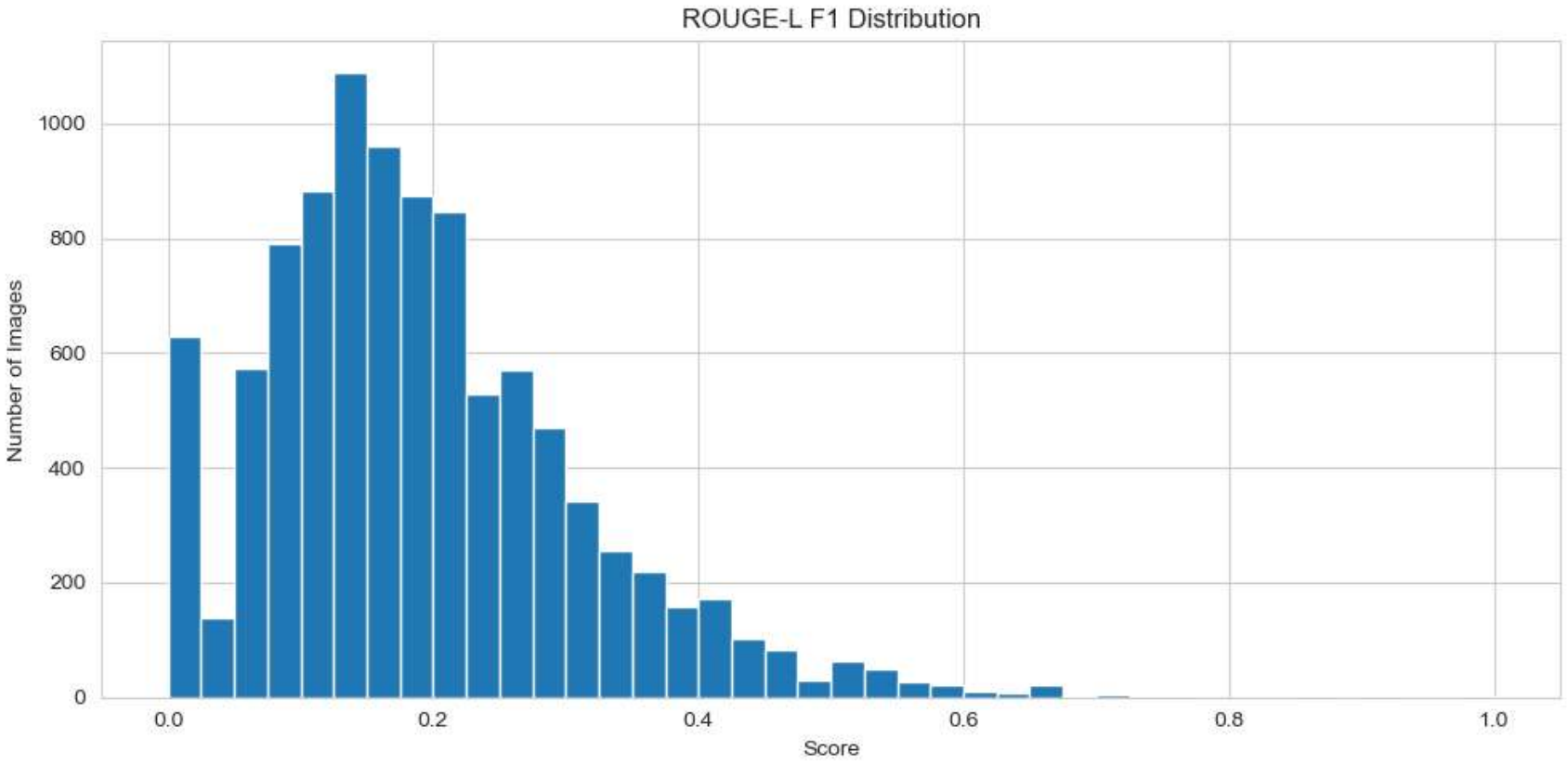
```
Mean ROUGE-L Scores
ROUGE-L Precision: 0.2610
ROUGE-L Recall: 0.1780
ROUGE-L F1: 0.1903
```

In [47]:

```
# display ROUGE-L score distributions
for metric in [
    "ROUGE-L Precision",
    "ROUGE-L Recall",
    "ROUGE-L F1"
]:
    plt.figure(figsize=(10, 5))
    plt.hist(ROUGE_results[metric], bins=40)
    plt.title(f"{metric} Distribution")
    plt.xlabel("Score")
    plt.ylabel("Number of Images")
    plt.tight_layout()
    plt.show()
```


ROUGE-L Precision Distribution
Number of Images
1000
800
600
400
200
0
0.0
0.2
0.4
0.6
0.8
1.0
Score


ROUGE-L Recall Distribution
Number of Images
Score
0
200
400
600
800
1000
1200
0.0
0.2
0.4
0.6
0.8
1.0



In [48]:
```
# display ROUGE-L results for each image
display(ROUGE_results)
```

	ID	ROUGE-L Precision	ROUGE-L Recall	ROUGE-L F1
0	ROCOv2_2023_test_000001	0.230769	0.300000	0.260870
1	ROCOv2_2023_test_000002	0.071429	0.125000	0.090909
2	ROCOv2_2023_test_000003	0.250000	0.294118	0.270270
3	ROCOv2_2023_test_000004	0.272727	0.153846	0.196721
4	ROCOv2_2023_test_000005	0.166667	0.142857	0.153846
...	...	...	...	...
9922	ROCOv2_2023_test_009968	0.142857	0.064516	0.088889
9923	ROCOv2_2023_test_009969	0.307692	0.085106	0.133333
9924	ROCOv2_2023_test_009970	0.166667	0.166667	0.166667
9925	ROCOv2_2023_test_009971	0.428571	0.052632	0.093750
9926	ROCOv2_2023_test_009972	0.428571	0.040541	0.074074

9927 rows × 4 columns

Meteor

In [49]:
```
# merge ground-truth and generated captions by image ID
METEOR_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
    how="inner",
    validate="many_to_one"
)
```

In [50]:
```
# tokenise captions consistently
def meteor_tokenize(caption):
    return re.findall(r"[a-z0-9]+", caption.lower())
```

In [51]:
```
# calculate METEOR score for each image
meteor_results = []
```

In [52]:
```
for image_id, group in METEOR_reference_generated_captions.groupby("ID"):

    # retrieve generated caption
    generated = group["generated_caption"].iloc[0]

    # retrieve and tokenise all ground-truth captions
    references = [
        meteor_tokenize(ref)
        for ref in group["cleaned_caption"]
        if isinstance(ref, str) and ref.strip()
    ]

    # calculate METEOR using the best matching reference
    if references and isinstance(generated, str) and generated.strip():
        score = meteor_score(
            references,
            meteor_tokenize(generated)
        )

        meteor_results.append({
            "ID": image_id,
            "METEOR": score
        })
```

In [53]:
```
# store results in DataFrame
METEOR_results = pd.DataFrame(meteor_results)
```

In [54]:
```
# display METEOR summary statistics
print(METEOR_results["METEOR"].describe())
```

```
count    9927.000000
mean        0.155531
std         0.127167
min         0.000000
25%         0.066372
50%         0.121951
75%         0.210044
max         0.996000
Name: METEOR, dtype: float64
```

In [55]:
```python
# display mean METEOR score
print(
    f"\nMean METEOR Score: "
    f"{METEOR_results['METEOR'].mean():.4f}"
)
```

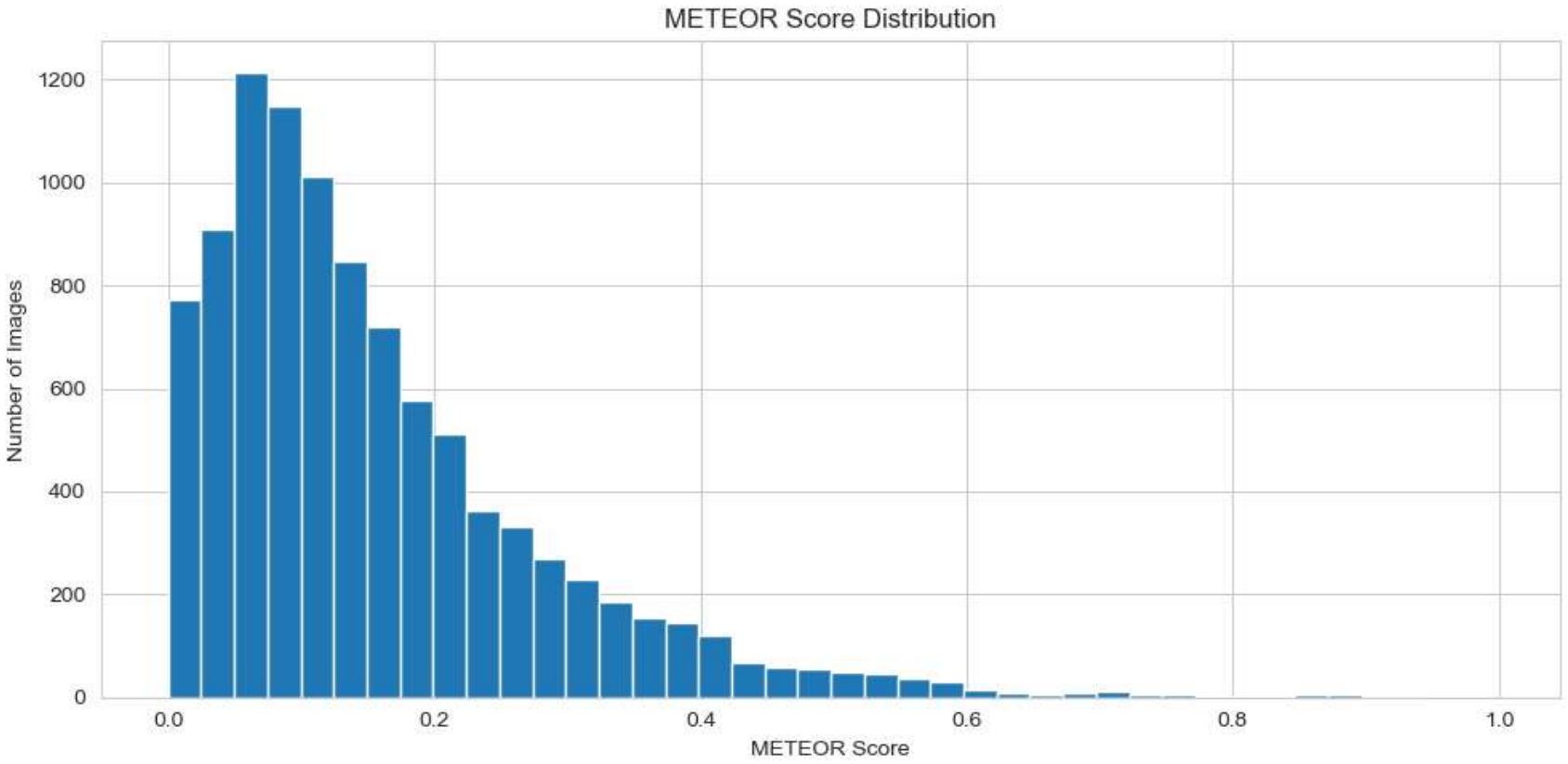
```
Mean METEOR Score: 0.1555
```

In [56]:
```python
# display METEOR score distribution
plt.figure(figsize=(10, 5))

plt.hist(
    METEOR_results["METEOR"],
    bins=40
)

plt.title("METEOR Score Distribution")
plt.xlabel("METEOR Score")
plt.ylabel("Number of Images")

plt.tight_layout()
plt.show()
```



In [57]:
```
# display METEOR results for each image
display(METEOR_results)
```

	ID	METEOR
0	ROCOv2_2023_test_000001	0.248112
1	ROCOv2_2023_test_000002	0.116279
2	ROCOv2_2023_test_000003	0.215029
3	ROCOv2_2023_test_000004	0.093834
4	ROCOv2_2023_test_000005	0.072464
...	...	...
9922	ROCOv2_2023_test_009968	0.051195
9923	ROCOv2_2023_test_009969	0.057339
9924	ROCOv2_2023_test_009970	0.083333
9925	ROCOv2_2023_test_009971	0.072115
9926	ROCOv2_2023_test_009972	0.037973

9927 rows × 2 columns

Caption Evaluation Using Cosine Similarity

In [58]:
```
# load pretrained sentence embedding model
sentence_model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2", device="cpu")
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits a
nd faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_
TOKEN to enable higher rate limits and faster downloads.
Loading weights:   0%|          | 0/103 [00:00<?, ?it/s]
```

In [59]:
```
# merge ground-truth and generated captions by image ID
CS_reference_generated_captions = pd.merge(
    test_set_captions,
    generated_captions,
    on="ID",
```

```
    how="inner",
    validate="many_to_one"
)
```

In [60]:
```
# verify every generated image has a reference caption
missing_ids = set(generated_captions["ID"]) - set(
    CS_reference_generated_captions["ID"]
)
```

In [61]:
```
# create sentence embeddings for ground-truth captions
reference_embeddings = sentence_model.encode(
    CS_reference_generated_captions["cleaned_caption"].tolist(),
    batch_size=64,
    normalize_embeddings=True,
    convert_to_numpy=True,
    show_progress_bar=True
)
```

```
Batches:   0%|          | 0/156 [00:00<?, ?it/s]
```

In [62]:
```
# create sentence embeddings for generated captions
generated_embeddings = sentence_model.encode(
    CS_reference_generated_captions["generated_caption"].tolist(),
    batch_size=64,
    normalize_embeddings=True,
    convert_to_numpy=True,
    show_progress_bar=True
)
```

```
Batches:   0%|          | 0/156 [00:00<?, ?it/s]
```

In [63]:
```
# calculate cosine similarity for corresponding captions
CS_reference_generated_captions["cosine_similarity"] = np.sum(
    reference_embeddings * generated_embeddings,
    axis=1
).clip(-1, 1)
```

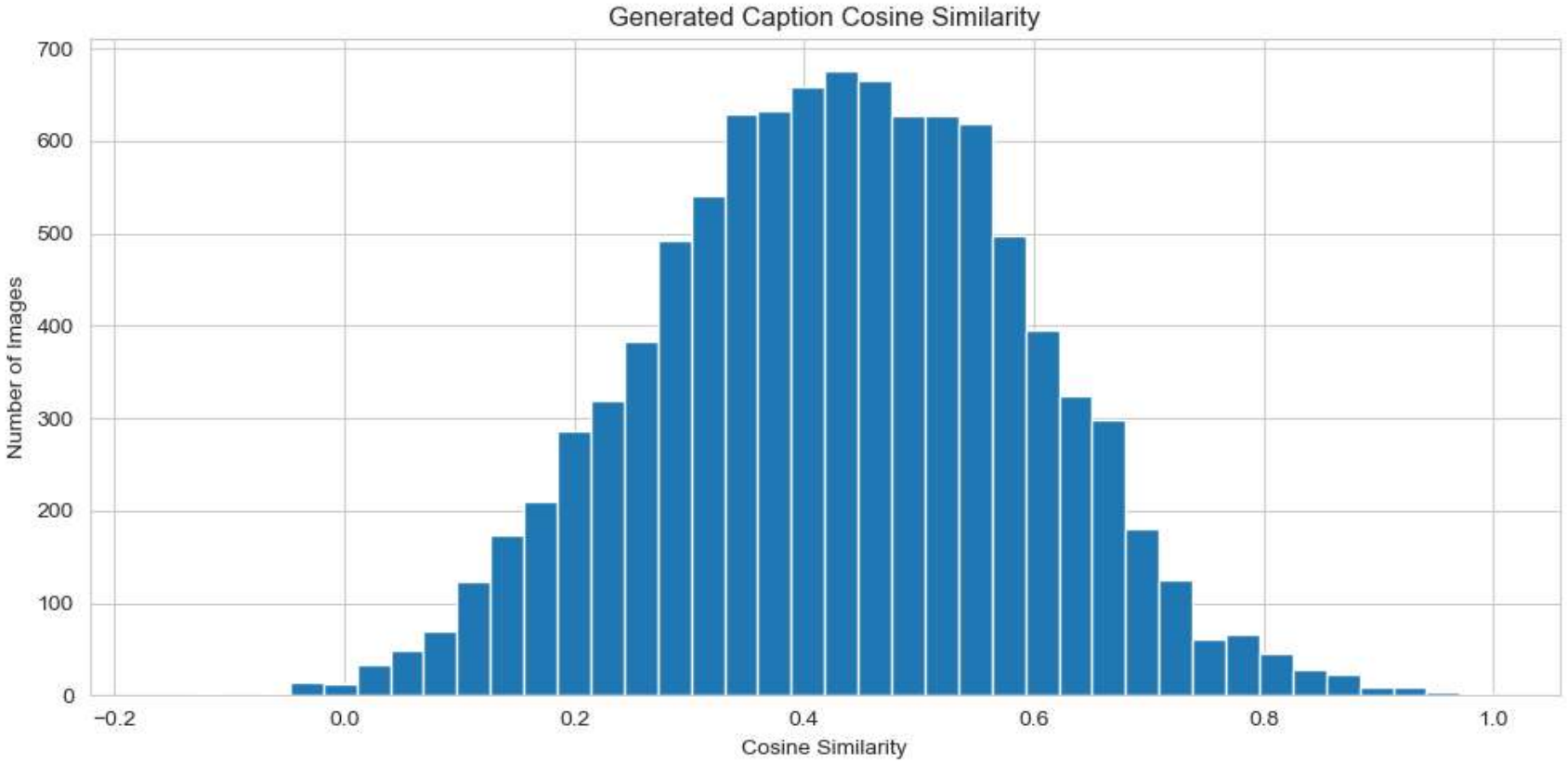
In [64]:
```
# retain best matching reference per image
CS_results = (
    CS_reference_generated_captions
    .groupby("ID", as_index=False)["cosine_similarity"]
```

```
    .max()
)
```

In [65]:
```
# display summary statistics
print(CS_results["cosine_similarity"].describe())
```

```
count    9927.000000
mean        0.430139
std         0.162817
min        -0.163967
25%         0.318443
50%         0.431610
75%         0.544503
max         1.000000
Name: cosine_similarity, dtype: float64
```

In [66]:
```
# display distribution
plt.figure(figsize=(10, 5))
plt.hist(
    CS_results["cosine_similarity"],
    bins=40
)
plt.title("Generated Caption Cosine Similarity")
plt.xlabel("Cosine Similarity")
plt.ylabel("Number of Images")
plt.tight_layout()
plt.show()
```



In [67]:
```
# display individual image results
display(CS_results)
```

	ID	cosine_similarity
0	ROCOv2_2023_test_000001	0.555744
1	ROCOv2_2023_test_000002	0.350259
2	ROCOv2_2023_test_000003	0.345109
3	ROCOv2_2023_test_000004	0.364864
4	ROCOv2_2023_test_000005	0.114432
...	...	...
9922	ROCOv2_2023_test_009968	0.373686
9923	ROCOv2_2023_test_009969	0.420453
9924	ROCOv2_2023_test_009970	0.459149
9925	ROCOv2_2023_test_009971	0.639870
9926	ROCOv2_2023_test_009972	0.670342

9927 rows × 2 columns

References

Hugging Face (2026) all-MiniLM-L6-v2: Pretrained Sentence Embedding Model. Available At https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

Natural Language Toolkit (NLTK) (2026) NLTK Python Package – BLEU and METEOR Evaluation. Available At https://pypi.org/project/nltk/

Natural Language Toolkit (NLTK) (2026) METEOR Score: API Documentation. Available At https://www.nltk.org/api/nltk.translate.meteor_score.html

Natural Language Toolkit (NLTK) (2026) Installing NLTK Data – WordNet and Linguistic Resources. Available At https://www.nltk.org/data.html

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

PyPI (2022) rouge-score: Python Implementation of ROUGE Evaluation Metrics. Available At https://pypi.org/project/rouge-score/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/

Sentence Transformers (2026) Sentence Transformers: Installation and Documentation. Available At https://www.sbert.net/docs/installation.html